

Dataset: California Housing

- 20,640 filas,
- 8 variables,
- 1 variable target: precio medio

Información sobre filas y columnas

```
> print(f"Filas: {df.shape[0]}, Columnas: {df.shape[1]}")
df.info()
✓ 0.0s

Filas: 20640, Columnas: 9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
8   MedHouseVal     20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB

print("ESTADÍSTICAS DESCRIPTIVAS")
print("="*50)
print(df.describe())
✓ 0.0s

ESTADÍSTICAS DESCRIPTIVAS
=====
count    MedInc    HouseAge    AveRooms    AveBedrms    Population \
mean      3.870671    28.639486     5.429000     1.096675    1425.476744
std       1.899822    12.585558     2.474173     0.473911    1132.462122
min       0.499900     1.000000     0.846154     0.333333     3.000000
25%       2.563400    18.000000     4.440716     1.006079     787.000000
50%       3.534800    29.000000     5.229129     1.048780    1166.000000
75%       4.743250    37.000000     6.052381     1.099526    1725.000000
max       15.000100    52.000000    141.909091    34.066667   35682.000000

count    AveOccup    Latitude    Longitude    MedHouseVal
mean      3.070655    35.631861   -119.569704     2.068558
std       10.386050     2.135952     2.003532     1.153956
min       0.692308    32.540000   -124.350000     0.149990
25%       2.429741    33.930000   -121.800000     1.196000
50%       2.818116    34.260000   -118.490000     1.797000
75%       3.282261    37.710000   -118.010000     2.647250
max      1243.333333    41.950000   -114.310000     5.000010
```

1. Ridge Regression

Es una regresión lineal con regularización, que busca la línea que mejor se ajusta a los datos

Hiperparámetro empleado: **alpha**

- alpha = 0.1: Poca regularización, modelo más flexible
- alpha = 1.0: Regularización moderada

2. Gradient Boosting

Construye muchos árboles de decisión de forma secuencial. Cada nuevo árbol intenta corregir los errores del anterior.

Hiperparámetros empleados: **n_estimators**, **max_depth** y **random_state**

- n_estimators = 100: Número de árboles a construir
- max_depth = 5: Profundidad máxima de cada árbol
- random_state = 42: Semilla para reproducibilidad

n Analogía: Es como un equipo de auditores donde cada uno revisa el trabajo del anterior y corrige errores.

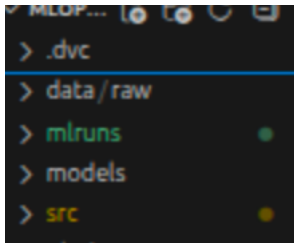
Procedimiento

1. El ejercicio ya estaba comenzado, y DVC iniciado desde antes. Así que en esta ocasión sólo volví a activar conda, entrar en la carpeta del challenge y abrir VS code.

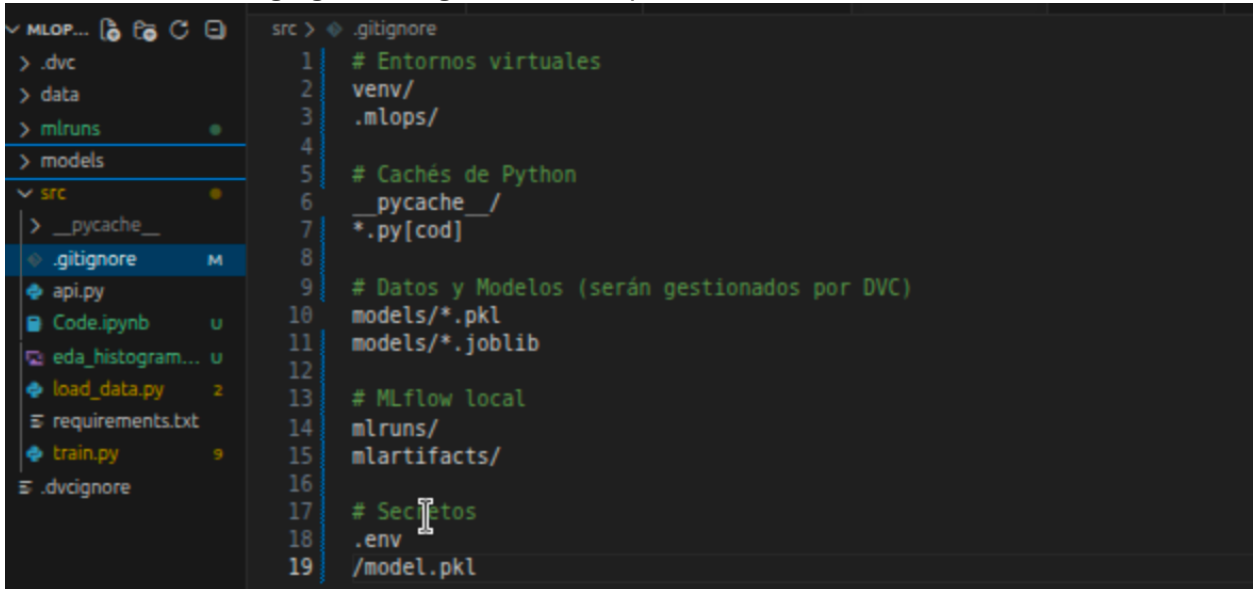
```
(base) mauricio@mauricio-VMware20-1:~/Escritorio$ conda activate production_mlops
(production_mlops) mauricio@mauricio-VMware20-1:~/Escritorio$ cd ~/Escritorio/mlops_challenge
(production_mlops) mauricio@mauricio-VMware20-1:~/Escritorio/mlops_challenge$ git init
dvc init
git add .
git commit -m "Inicializar proyecto"
Reinicializado el repositorio Git existente en /home/mauricio/Escritorio/mlops_challenge/.git/
ERROR: failed to initiate DVC - '.dvc' exists. Use '-f' to force.
En la rama main
Tu rama está actualizada con 'origin/main'.

nada para hacer commit, el árbol de trabajo está limpio
(production_mlops) mauricio@mauricio-VMware20-1:~/Escritorio/mlops_challenge$ code .
(production_mlops) mauricio@mauricio-VMware20-1:~/Escritorio/mlops_challenge$
```

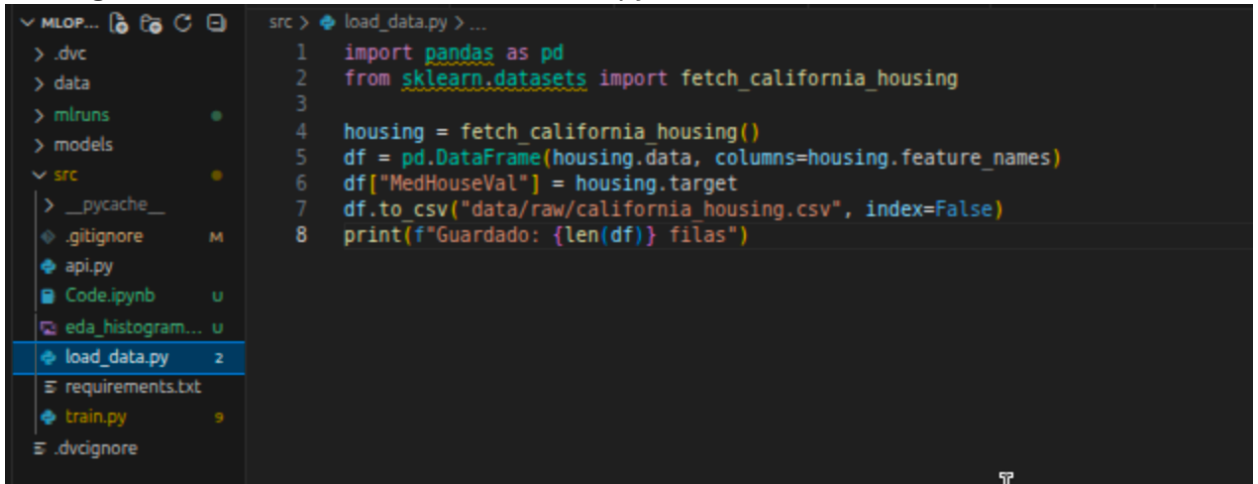
2. Una vez que estaba dentro de VS Code, cree las carpetas data, mlruns, models, src



3. Se crea un fichero .gitignore colgado de la carpeta raíz



4. Se cargan los datos dentro de src/load_data.py



5. Versionamos los datos

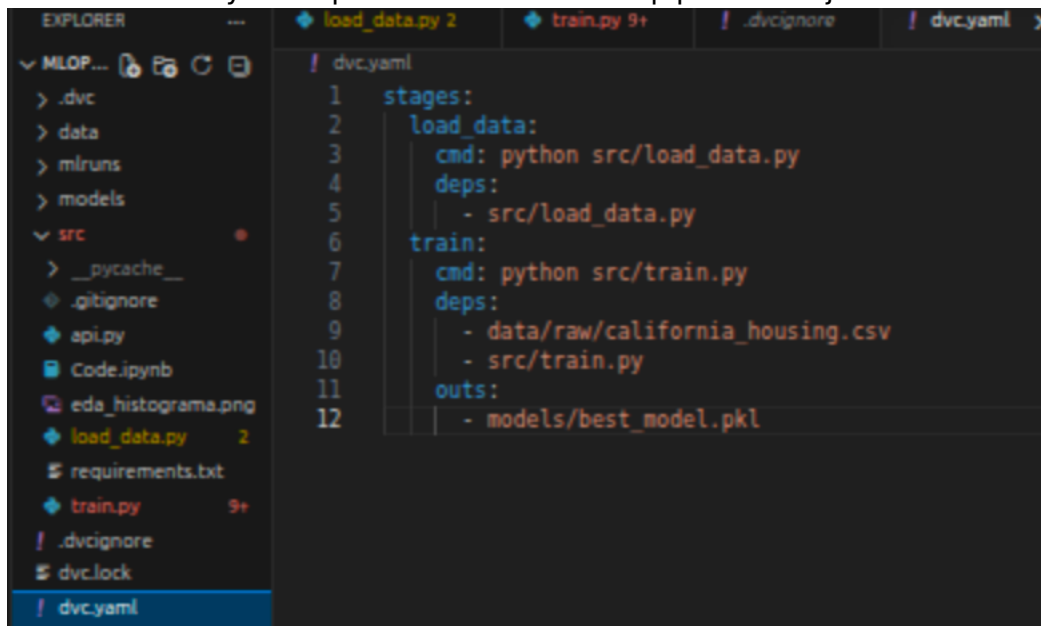
```
• (base) mauricio@mauricio-VWare20-1:~/Escritorio/mlops_challenge$ cd ~/Escritorio/mlops_challenge
conda activate production_mlops
dvc push
Collecting
Pushing
1 file pushed
○ (production_mlops) mauricio@mauricio-VWare20-1:~/Escritorio/mlops_challenge$
```

6. Creamos src/train.py y agregamos nuestros modelos (Ridge y Gradient Boosting Regression) para entrenarlos. En este caso hicimos el manual logging de MLflow, usando log_param() o log_metric(). las métricas que estamos empleando son rmse, mae y r2

```
1 import pandas as pd
2 import numpy as np
3 import mlflow
4 import mlflow.sklearn
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.linear_model import Ridge
8 from sklearn.ensemble import GradientBoostingRegressor
9 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
10 import pickle
11 import os
12
13 mlflow.set_tracking_uri("file:./mlruns")
14 mlflow.set_experiment("california-housing-prediction")
15
16 # Cargar datos
17 df = pd.read_csv("data/raw/california_housing.csv")
18 X = df.drop("MedHouseVal", axis=1)
19 y = df["MedHouseVal"]
20 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
21
22 # Modelos a probar
23 experiments = [
24     (Ridge(alpha=0.1), "Ridge_0.1", {"alpha": 0.1}),
25     (Ridge(alpha=1.0), "Ridge_1.0", {"alpha": 1.0}),
26     (GradientBoostingRegressor(n_estimators=100, max_depth=5, random_state=42), "GB_100_5", {"n_estimators": 100, "max_depth": 5}),
27 ]
28
29 best_model, best_scaler, best_r2, best_name = None, None, -999, ""
30
31 for model, name, params in experiments:
32     scaler = StandardScaler()
33     X_train_s = scaler.fit_transform(X_train)
34     X_test_s = scaler.transform(X_test)
35
36     model.fit(X_train_s, y_train)
37     y_pred = model.predict(X_test_s)
38
39     rmse = np.sqrt(mean_squared_error(y_test, y_pred))
40     mae = mean_absolute_error(y_test, y_pred)
41     r2 = r2_score(y_test, y_pred)
42
43     with mlflow.start_run(run_name=name):
44         for k, v in params.items():
45             mlflow.log_param(k, v)
46         mlflow.log_metric("rmse", rmse)
47         mlflow.log_metric("mae", mae)
48         mlflow.log_metric("r2", r2)
49         mlflow.sklearn.log_model(model, "model")
50
51     print(f"(name): RMSE={rmse:.4f}, MAE={mae:.4f}, R²={r2:.4f}")
52
53     if r2 > best_r2:
54         best_r2, best_model, best_scaler, best_name = r2, model, scaler, name
55
56 os.makedirs("models", exist_ok=True)
57 with open("models/best_model.pkl", "wb") as f:
58     pickle.dump({"model": best_model, "scaler": best_scaler}, f)
59
60 print(f"\n🏆 Mejor: {best_name} (R²={best_r2:.4f})")
```

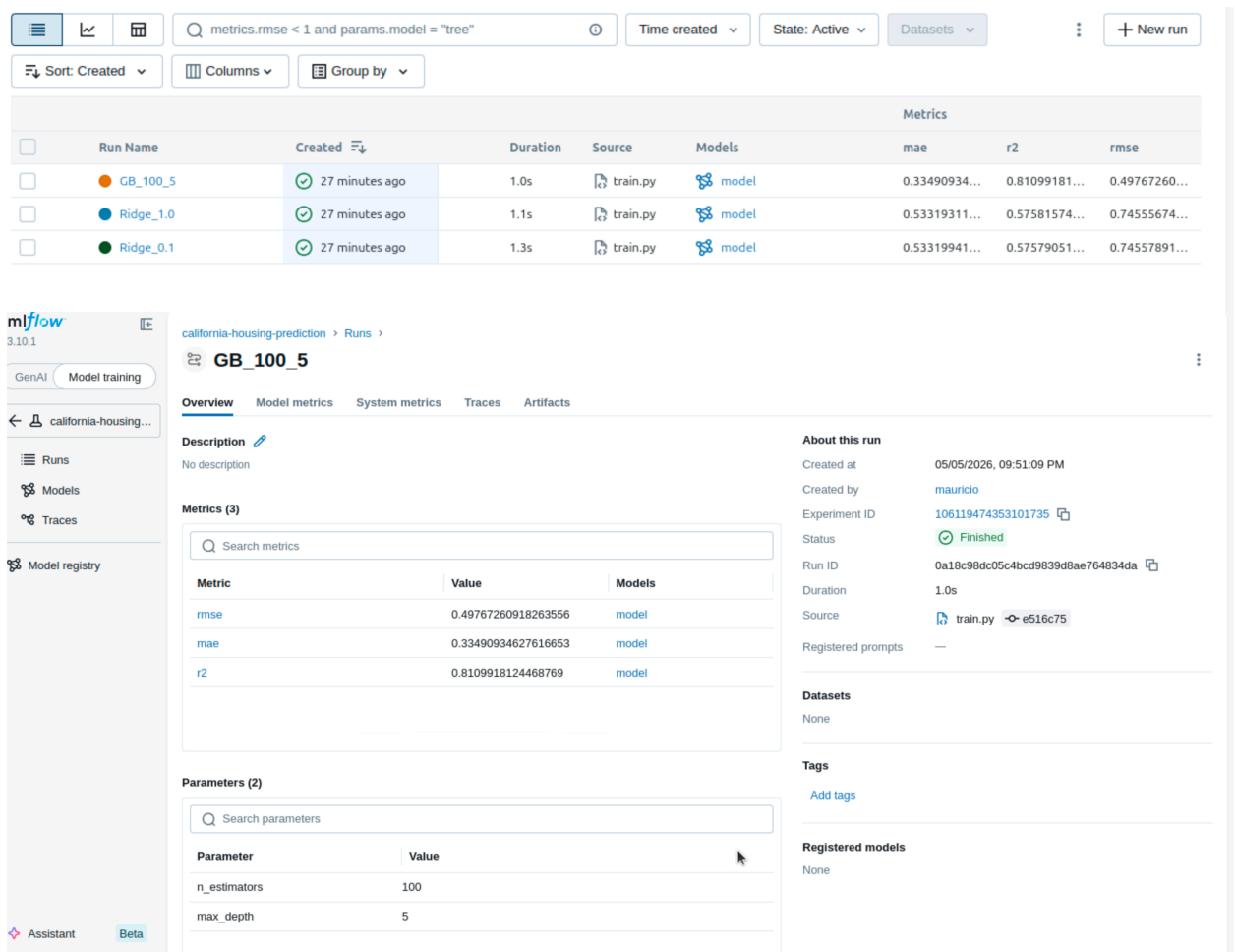
```
with mlflow.start_run(run_name=name):
    for k, v in params.items():
        mlflow.log_param(k, v)
    mlflow.log_metric("rmse", rmse)
    mlflow.log_metric("mae", mae)
    mlflow.log_metric("r2", r2)
    mlflow.sklearn.log_model(model, "model")
```

7. Creamos el dvc.yaml – que sirve como nuestro pipeline de ejecución del modelo



```
1 stages:
2   load_data:
3     cmd: python src/load_data.py
4     deps:
5       - src/load_data.py
6   train:
7     cmd: python src/train.py
8     deps:
9       - data/raw/california_housing.csv
10      - src/train.py
11     outs:
12       - models/best_model.pkl
```

8. Ejecutamos y abrimos MLFlow, aquí podemos ver nuestros 3 modelos y los resultados de cada uno



The top part of the screenshot shows a table of runs in MLFlow. The table has columns for Run Name, Created, Duration, Source, Models, and Metrics (mae, r2, rmse). There are three runs listed: GB_100_5, Ridge_1_0, and Ridge_0_1.

Run Name	Created	Duration	Source	Models	mae	r2	rmse
GB_100_5	27 minutes ago	1.0s	train.py	model	0.33490934...	0.81099181...	0.49767260...
Ridge_1_0	27 minutes ago	1.1s	train.py	model	0.53319311...	0.57581574...	0.74555674...
Ridge_0_1	27 minutes ago	1.3s	train.py	model	0.53319941...	0.57579051...	0.74557891...

The bottom part of the screenshot shows the detailed view of the run 'GB_100_5'. It includes a description, metrics, parameters, and other run details.

Run Name: GB_100_5

Overview | Model metrics | System metrics | Traces | Artifacts

Description
No description

Metrics (3)

Metric	Value	Models
rmse	0.49767260918263556	model
mae	0.33490934627616653	model
r2	0.8109918124468769	model

Parameters (2)

Parameter	Value
n_estimators	100
max_depth	5

About this run

Created at: 05/05/2026, 09:51:09 PM
Created by: mauricio
Experiment ID: 106119474353101735
Status: Finished
Run ID: 0a18c98dc05c4bcd9839d8ae764834da
Duration: 1.0s
Source: train.py
Registered prompts: —

Datasets
None

Tags
Add tags

Registered models
None

Mauricio Orozco

MLOps challenge

The image displays two screenshots of the mlflow web interface, showing the 'Overview' tab for a model named 'Ridge_1.0' (top) and 'Ridge_0.1' (bottom). Both screenshots show the 'Metrics (3)' section with a table of performance metrics.

Top Screenshot (Ridge_1.0):

Metric	Value	Models
rmse	0.7455567442814779	model
mae	0.5331931195789733	model
r2	0.5758157428913684	model

Bottom Screenshot (Ridge_0.1):

Metric	Value	Models
rmse	0.7455789118982766	model
mae	0.5331994146968932	model
r2	0.5757905180002313	model

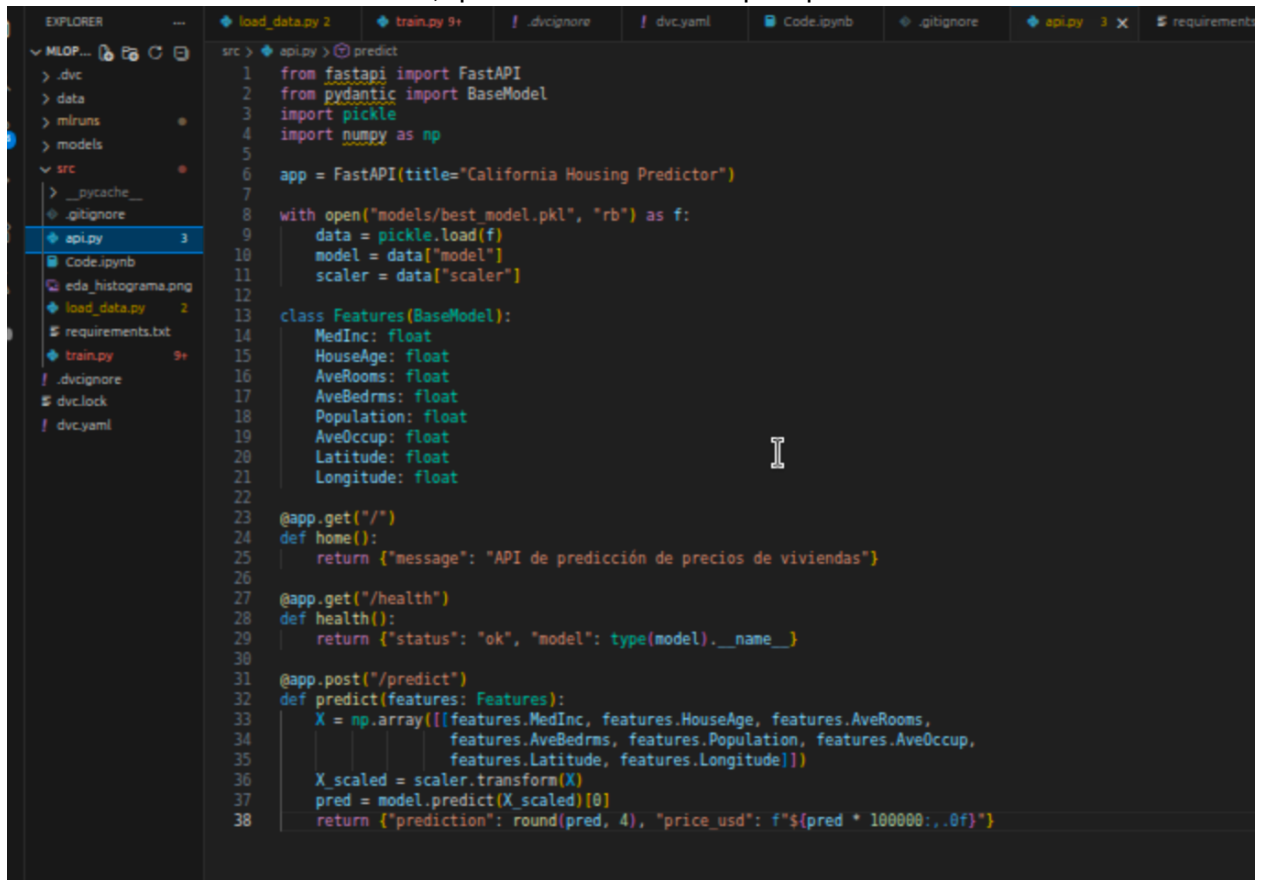
The bottom screenshot also shows the 'Parameters (1)' section with a table:

Parameter	Value
alpha	0.1

Como podemos ver, el modelo Gradient Boosting con $n_estimators=100$ y $max_depth=5$ fue el que obtuvo el mejor rendimiento con un R^2 de 0.811, explicando el 81% de la variación en precios, frente al 57% de los modelos Ridge. Esto se debe a que Gradient Boosting captura relaciones no lineales entre las variables, mientras que Ridge asume relaciones lineales simples. Consideramos R^2 porque es el que explica el porcentaje de la variación en precios.

Mauricio Orozco
MLOps challenge

9. Para poner el modelo en producción, creamos una API con FASTAPI, cargamos el mejor modelo y utilizamos un endpoint para hacer la predicción en base a las características de una vivienda, que deberían ser el input que le damos nosotros

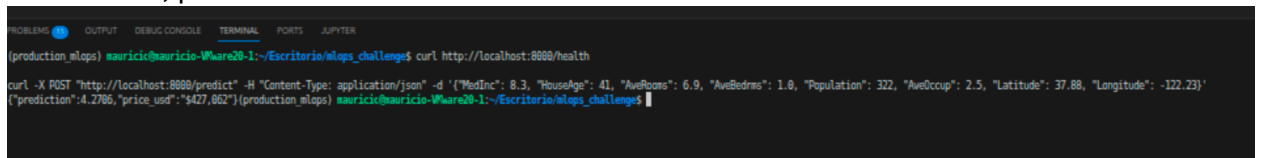


```
EXPLORER  src > api.py 2  train.py 9+  .dvcignore  dvc.yaml  Code.ipynb  .gitignore  api.py 3 x  requirements.txt

MLOP...  .dvc  > data  > mlruns  > models  > src  > __pycache__  .gitignore  api.py 3  Code.ipynb  eda_histograma.png  load_data.py 2  requirements.txt  train.py 9+  .dvcignore  dvc.lock  dvc.yaml

src > api.py > predict
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  import pickle
4  import numpy as np
5
6  app = FastAPI(title="California Housing Predictor")
7
8  with open("models/best_model.pkl", "rb") as f:
9      data = pickle.load(f)
10     model = data["model"]
11     scaler = data["scaler"]
12
13 class Features(BaseModel):
14     MedInc: float
15     HouseAge: float
16     AveRooms: float
17     AveBedrms: float
18     Population: float
19     AveOccup: float
20     Latitude: float
21     Longitude: float
22
23 @app.get("/")
24 def home():
25     return {"message": "API de predicción de precios de viviendas"}
26
27 @app.get("/health")
28 def health():
29     return {"status": "ok", "model": type(model).__name__}
30
31 @app.post("/predict")
32 def predict(features: Features):
33     X = np.array([[features.MedInc, features.HouseAge, features.AveRooms,
34                   features.AveBedrms, features.Population, features.AveOccup,
35                   features.Latitude, features.Longitude]])
36     X_scaled = scaler.transform(X)
37     pred = model.predict(X_scaled)[0]
38     return {"prediction": round(pred, 4), "price_usd": f"${pred * 100000:.0f}"}
```

10. Finalmente, probamos la API usando curl



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER

(production_mlops) mauricio@mauricio-Ware20-1:~/Escritorio/ml_ops_challenge$ curl http://localhost:8080/health
{"message": "API de predicción de precios de viviendas"}

(production_mlops) mauricio@mauricio-Ware20-1:~/Escritorio/ml_ops_challenge$ curl -X POST "http://localhost:8080/predict" -H "Content-Type: application/json" -d '{"MedInc": 8.3, "HouseAge": 41, "AveRooms": 6.9, "AveBedrms": 1.6, "Population": 322, "AveOccup": 2.5, "Latitude": 37.88, "Longitude": -122.23}'
{"prediction": 4.2786, "price_usd": "$427,862"}(production_mlops) mauricio@mauricio-Ware20-1:~/Escritorio/ml_ops_challenge$
```